DSAI 490 Assignment 2

DO NOT PROVIDE THE DOCUMENT TO AI TO SUGGEST A MODEL FOR SOLVING THE PROBLEM OR TO EXPLAIN THE PROBLEM. YOU MUST UNDERSTAND AND SOLVE IT ON YOUR OWN, OR YOU WILL RECEIVE A ZERO. AI MAY ONLY ASSIST WITH CODING. INSTRUCT THE AI NOT TO SUGGEST OR INTERFERE WITH MODEL SELECTION AT ALL, AND DO NOT USE IT TO EXPLAIN THE PROBLEM.

1. Coding Problem : Dates Generator

Motivation:

Deep Generative models are a well known tool in deep learning to produce synthetic high quality data similar to data seen during train time –i.e. fake data that looks real. You may have come across famous ***unconditional*** generation examples, like this-person-does-not-exist.com which shows high quality **randomly** generated photos of human faces using a type of generative models called GANs, or possibly the more involved diffusion models. If we could utilize the same kind of models to do a more deliberate (designed, **conditioned**) generation of faces, we can do amazingly fun things like make someone look younger or older – possibilities are limitless.

Problem Description:

For this problem, you are asked to generate a date given a set of conditions, using any neural network architecture you would like. Your input (x) is the conditions on the date, and the output (y) is ANY date that complies with those conditions. This means that, like any generative model, **there are many right answers per input x.**

Minimum number of models to be implemented is four: two from the course (One of them should be GAN) and two from outside the course.

Dataset

You are given a **data.txt** file containing the entire dataset, one entry per line. Each line is in the format:

```
[day condition] [month condition] [leap year condition] [decade condition] date
```

Some examples:

```
[MON] [DEC] [False] [196] 3-12-1962
[THU] [DEC] [True] [204] 3-12-2048
[WED] [JAN] [False] [181] 10-1-1810
```

- **day condition: input.** A three letter token, with square brackets around it, depicting the day that the generated date should match. Eg: `[WED]` means that the output date should occur on a Wednesday. For this condition to be **PASSED**, the date has to match a wednesday in any month or year.
- **month condition: input.** A three letter token, with square brackets around it, depicting the month that the output date should occur in. Eg: `[JAN]` means that the output date should be in January. For this condition to be **PASSED**, the date has to occur in January, of any year and on any day.
- **leap year condition: input.** Either a `[False]` or a `[True]` token, depicting whether the output year should be a leap year (True) or not (False). For this condition to be **PASSED**, the date has to occur in a leap year, regardless of the decade, month, or day. Leap year definition can be found here.
- **decade condition: input.** A three letter token, with square brackets around it, depicting the decade that the output date should occur in. Eg: `[192]` means that the output date can be any date from `1-1-1920` to `31-12-1929`. For this condition to be **PASSED**, the date has to occur in the given decade, regardless of the day, month, or being in a leap year.
- **date: output.** The only output. A date string of the format `dd-mm-yyyy` (day, month, year). It should match all the previous conditions.

Provided Files:

1. `data.txt :` all data
2. `example_input.txt :` example file with input conditions only, without output.

Hints:

1. The hardest part about this problem will probably be figuring out the **problem formulation** and corresponding network **architecture**. It is an important step for any deep learning project. Take your time reading the problem description and reading about possible models online.
2. It is better to implement a **custom tokenizer**, and maybe effective to change the order of the tokens for some architectures. Think about this: What are easier tokens to figure out given the input conditions ? digit by digit, in the date.
3. **Data imbalance:** For some conditions, there is much less data than others. You can handle this using data imbalance handling techniques. It is not essential, but will improve your accuracy. Do not start with it, but maybe finish with it.
4. This is a generation, not a classification problem. Accuracy is not the best way to monitor your model output since several answers are correct. What is a better way you can use to monitor your model while training ?

Requirements:

1. A private cloned repository containing your solution
2. A pdf or .docx named "`Assignment_1_your_name_your_id`" file containing briefing about your methodology, your analysis of the outputs, training and test loss graphs, and any figures you see relevant.

How to:

1. Your repository is expected to have:
    a. The model's training, inference, and evaluation code in a folder called "model". It should also contain the model's training weights.
    b. The inference code should be located in a file of path: `repo/model/predict.py`, and can be run using the following command: `python predict.py -i $path_to_input_file -o $path_to_output_file` , producing a file with the predictions in `$path_to_output_file`. An example input file is present under the path `repo/data/example_input.txt`. The output format should match the `data.txt` format exactly (conditions + output), in the same order as the `example_input.txt`.

2. conda environment spec file to allow replication of your results if needed 4.

"`Assignment1_your_name_your_id`" report file is an important part of the grading, make sure to reflect on your choices and analysis briefly. The document should not be more than 5 pages by any means, one page could be good enough. **Think of it as a document to communicate your implementation (without the code), and why it works, to a fellow deep learning engineer.**

Constraints:

1. If you are going to use TensorFlow 2.x, you should implement the training loop from scratch **(Do NOT use model.fit())**
   Hint: you can use tf.GradientTape() to record the operations and get the gradients (Check this link).
2. Your code is required to only work for dates in the range [`1-1-1800` to `31-12-2200`]. Do not overwhelm yourself by covering all possible dates in history.

Evaluation Criteria:

1. **Evaluation Metric**

2. Report: Briefness & Clarity – Can we make the same conclusions you made, fast ? 3. **Problem formulation** (make sure you explain it well in the report file, including tokenization, loss function, architecture ..etc)

4. **Code readability:** This is necessary for us to be able to evaluate your code. Do not submit all your code in one notebook. You are a software engineer, structure your code in folders, files, and classes when needed.
5. **Code Correctness:** Your logic should be sane, assumptions you make should be explained in the report.
6. **Results readability:** Can we visually and logically validate that your results are ok ? Provide some output examples, maybe also provide examples where your model failed and your reflection on them.
7. **Percentage of original code**: While not essential, it is important to be able to see your coding quality and skills.
8. **Bonus:** Deep Learning and coding best practices: shuffling data, test data, manual seed to allow for experiment replication, type hinting ..etc.